



ELABORAÇÃO DE ALGORITMOS

01 - Crie uma função chamada somar que recebe dois números inteiros como parâmetros e retorna a soma deles. No programa principal, leia dois inteiros digitados pelo usuário, chame a função e exiba o resultado.

02 - Implemente uma função quadrado que receba um número inteiro e retorne o seu quadrado. Na *main*, leia um valor e mostre o quadrado utilizando a função.

03 - Construa uma função ehPar que receba um número inteiro e retorne 1 se ele for par e 0 se for ímpar (utilize uma estrutura condicional). Na *main*, leia um número e use a função para informar se ele é par ou ímpar.

04 - Crie uma função maiorEntreDois que receba dois números reais (*float*) e retorne o maior deles. Na *main*, leia dois valores e exiba o maior usando a função.

05 - Faça uma função celsiusParaFahrenheit que receba uma temperatura em graus Celsius (*float*) e retorne a temperatura convertida em Fahrenheit. Fórmula: $F = C * 1.8 + 32$. Na *main*, leia uma temperatura e mostre a conversão.

06 - Implemente uma função media que receba três notas (*float*) e retorne a média aritmética. Na *main*, leia as três notas e exiba a média.

07 - Crie uma função areaRetangulo que receba a base e a altura de um retângulo (*float*) e retorne sua área. Na *main*, leia os valores e mostre a área calculada pela função.

08 - Construa uma função imprimirTabuada que receba um número inteiro e imprima sua tabuada de 1 a 10 (utilize uma estrutura de repetição). A função não precisa retornar valor (*void*). Na *main*, leia o número e chame a função.

09 - Implemente uma função contagemRegressiva que receba um número inteiro *n* e imprima na tela os valores de *n* até 0, um por linha (use laço de repetição). Função sem retorno. Na *main*, leia *n* e chame a função.

10 - Crie uma função `classificarNumero` que receba um número inteiro e imprima na tela uma das mensagens: "Positivo", "Negativo" ou "Zero", utilizando condicionais. Na *main*, leia o número e chame a função.

11 - Implemente uma calculadora simples utilizando modularização. O programa deve possuir, no mínimo, quatro funções: somar, subtrair, multiplicar e dividir, cada uma recebendo dois números reais e retornando o resultado da operação. Na *main*:

- Exiba um menu com as opções (1-Somar, 2-Subtrair, 3-Multiplicar, 4-Dividir, 0-Sair).
- Leia a opção escolhida e os dois números.
- Utilize estrutura condicional (*switch* ou *if/else*) para chamar a função correspondente.
- Repita o menu até que o usuário escolha sair (estrutura de repetição).
- Trate divisão por zero exibindo uma mensagem de erro.

12 - Crie uma função `ehPrimo` que receba um número inteiro positivo e retorne 1 se o número for primo e 0 caso contrário (utilize laço e condicional dentro da função). No programa principal, leia uma quantidade *N* e, em seguida, leia *N* números inteiros, chamando a função para cada um deles e exibindo se cada número é primo ou não.

13 - Desenvolva um programa que leia as notas (*float*) de uma turma de *N* alunos (*N* informado pelo usuário) e calcule estatísticas básicas. Crie funções separadas para:

- `lerNotas`: lê *N* notas e armazena em um vetor.
- `calcularMedia`: recebe o vetor e o tamanho, retorna a média da turma.
- `maiorNota`: recebe o vetor e o tamanho, retorna a maior nota.
- `menorNota`: recebe o vetor e o tamanho, retorna a menor nota.
- `contarAprovados`: recebe o vetor, o tamanho e a nota mínima (ex.: 6.0); retorna o número de alunos aprovados.

Na *main*, leia *N*, chame `lerNotas` e em seguida exiba média, maior, menor nota e número de aprovados.

14 - Crie um programa que calcule a combinação simples $C(n,k) = n! / (k! * (n-k)!)$. Para isso, implemente:

- Função `fatorial(int n)` que retorne *n!* usando estrutura de repetição.
- Função `combinacao(int n, int k)` que utilize a função fatorial para calcular $C(n,k)$.
- Função `validarEntrada(int n, int k)` que retorne 1 se *n* e *k* são valores válidos ($n \geq 0$, $k \geq 0$ e $k \leq n$) e 0 caso contrário.

Na *main*, leia n e k , valide os valores com `validarEntrada` e, se válidos, mostre o resultado de $C(n,k)$. Caso contrário, exiba uma mensagem de erro. Permita repetir o cálculo enquanto o usuário desejar (uso de repetição).

15 - Desenvolva um jogo de adivinhação utilizando modularização. O programa deve sortear um número inteiro entre 1 e 100 e permitir que o usuário tente adivinhá-lo. Implemente as seguintes funções:

- `sortearNumero`: retorna um número aleatório entre 1 e 100 (utilize `rand()` e `srand(time(NULL))`).
- `lerPalpite`: lê e retorna o palpite do usuário.
- `verificarPalpite`: recebe o palpite e o número sorteado e retorna -1 (palpite menor), 0 (acertou) ou 1 (palpite maior).
- `exibirDica`: recebe o resultado da verificação e imprime uma mensagem adequada ("Tente um número maior", "Tente um número menor" ou "Parabéns, você acertou!").
- `jogarNovamente`: pergunta ao usuário se deseja jogar novamente e retorna 1 (sim) ou 0 (não).

Na *main*, controle todo o fluxo do jogo: sorteio, leitura de palpites em laço até acertar, contagem do número de tentativas, exibição do total ao final e possibilidade de jogar novamente. Utilize repetições e condicionais conforme necessário.